

JAVA — MODULE 1: INTRODUCTION TO JAVA

Overview of Java, its features, environment setup, first Java program, PATH/CLASSPATH, and JVM architecture.

Importance of Java for Career

- **High Demand** — used by Google, Amazon, Oracle, IBM, TCS, Infosys, Wipro, Accenture
- **Job Roles** — Java Developer, Backend Developer, Full Stack Developer, Android Developer, Software Engineer
- **Foundation for DSA** — widely used to learn Data Structures & Algorithms for placements
- **Enterprise Development** — used by banks, government, and multinational companies
- **Android Development** — one of the primary languages for Android apps
- **Good Salary Potential and Strong Community Support**
- **Future Scope** — cloud computing, microservices, fintech, big data

1. Introduction to Java

Java is a high-level, object-oriented, class-based, platform-independent language developed by James Gosling and team at Sun Microsystems (1995). Now owned by Oracle Corporation.

Philosophy: **“Write Once, Run Anywhere (WORA)”** — a program written on one OS runs on another OS without modification, as long as a JVM is installed.

Java is simple, secure, reliable, portable, and widely used in industry.

2. History of Java

Year	Event
1991	Java project started by James Gosling
1995	Java officially released by Sun Microsystems
2009	Oracle acquired Sun Microsystems
Present	Java is maintained by Oracle

Java was initially called **Oak** (after an Oak tree outside Gosling's office), later renamed Java.

3. Applications of Java

Category	Examples	Technologies
Desktop Applications	Calculator, Notepad, Banking Software	Java Swing, JavaFX
Web Applications	Banking websites, E-commerce websites	Servlet, JSP, Spring Boot, Hibernate
Mobile Applications	WhatsApp, Uber (Android)	Java, Kotlin
Enterprise Applications	Banking systems, Railway reservation, Hospital mgmt.	Secure & scalable
Scientific Applications	Simulations, data analysis	—
Cloud Computing	AWS, Google Cloud, Azure	—
Game Development	Minecraft (Java Edition)	—

4. Features of Java

Simple

Syntax similar to C/C++. Removes pointer arithmetic, multiple inheritance via classes, and operator overloading.

```
System.out.println("Hello world");
```

Object-Oriented

Based on classes and objects. OOP Principles: Encapsulation, Inheritance, Polymorphism, Abstraction.

```
class Student { }
```

Platform Independent

Source code is compiled to bytecode; the JVM executes that bytecode on the target OS — so code written on Windows can run on Linux/macOS unchanged.

Java Code → Bytecode (.class) → JVM → Operating System

Portable

Bytecode can be moved between computers; runs wherever a JVM exists.

Secure

- No direct pointer access
- Bytecode verification
- Security Manager (legacy)
- Automatic memory management

Robust

Strong & reliable — via Exception Handling, Garbage Collection, strong memory management.

Multithreaded

Supports multiple threads, e.g. a browser downloading, playing music, and loading images together.

Distributed

Supports networking — client-server apps, web services, distributed systems.

High Performance

Uses a JIT (Just-In-Time) Compiler, converting bytecode to machine code during execution.

Dynamic

Supports dynamic class loading and runtime memory allocation.

Architecture Neutral

Bytecode is independent of processor architecture — runs on Intel, AMD, ARM.

5. Setting Up Java Environment

- **Step 1:** Download the latest JDK from Oracle or an OpenJDK distribution
- **Step 2:** Install JDK by running the installer
- **Step 3:** Set JAVA_HOME (e.g. C:\Program Files\Java\jdk-21) and add %JAVA_HOME%\bin to PATH
- **Step 4:** Verify with `java -version` and `javac -version`

6. PATH and CLASSPATH Variables

PATH Variable

Tells the OS where to find Java executables (java, javac), so commands run from any directory without typing the full JDK path.

Example: C:\Program Files\Java\jdk-21\bin

CLASSPATH Variable

Tells the JVM/compiler where to find .class files and external .jar libraries — for user-defined classes, JARs, and packages.

Example: C:\Project\classes;C:\Libraries\mysql.jar;. (. = current directory)

PATH vs CLASSPATH

PATH	CLASSPATH
Finds Java executables (java, javac)	Finds Java classes (.class) and JAR files
Used by the Operating System	Used by JVM and Java Compiler
Points to the JDK bin folder	Points to class folders and library files

Trick: PATH = where Java commands live. CLASSPATH = where Java classes/libraries live.

7. How Java Works

Source code (.java) → Compiler (javac) → Bytecode (.class) → JVM (Interpreter + JIT) → Machine Code → Output

- **Source Code** — written by the programmer
- **Compiler (javac)** — converts source code into bytecode

- **Bytecode** — platform-independent intermediate code
- **JVM** — converts bytecode into machine code
- **Machine Code** — executed by the CPU

One-line exam answer: Java works by compiling source code into bytecode, which is then executed by the JVM by converting it into machine code.

8. Writing & Running the First Java Program

File: Hello.java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Explanation

- **public class Hello** — public: accessible anywhere; class: defines a class; Hello: class name (must match filename)
- **public static void main(String[] args)** — static: callable without an object; void: returns nothing; main: entry point; String[] args: command-line arguments
- **System.out.println()** — System: built-in class; out: standard output stream; println(): prints text with a newline

Compile & Run

```
javac Hello.java → generates Hello.class
java Hello → runs the program (no .class extension)
```

Compilation flow: Hello.java → javac → Hello.class (Bytecode) → JVM → Machine Code → Output

9. JVM (Java Virtual Machine)

JVM executes Java bytecode (.class files) and acts as a bridge between the Java program and the OS. It makes Java platform-independent by interpreting the same bytecode on different operating systems.

Java code is not directly converted to machine code — javac compiles to bytecode, and the JVM converts that bytecode to machine code for the OS to execute.

JDK, JRE, JVM Relationship

```
JDK → contains JRE + Java Compiler (javac) + development tools
JRE → contains JVM + Java Class Libraries (no compiler)
```

JDK	JRE
Used to write, compile, and run Java programs	Used only to run Java programs
Includes javac, jar, javadoc, and JRE	Includes JVM and class libraries
Converts .java files into .class files	Cannot compile Java source code

10. JVM Architecture

1. Class Loader Subsystem

Loads .class files into memory.

- **Loading** — via Bootstrap, Extension (Platform), and Application Class Loaders
- **Linking** — Verify (checks bytecode) → Prepare (allocates memory) → Resolve (references)
- **Initialization** — initializes static variables and runs static blocks

2. Runtime Data Area

- **Method Area** — class info, methods, static variables
- **Heap** — all objects and arrays (shared by all threads)
- **Stack** — local variables and method calls (one per thread)
- **PC Register** — address of the current instruction per thread
- **Native Method Stack** — data for native (C/C++) methods

3. Execution Engine

- **Interpreter** — executes bytecode instruction by instruction
- **JIT Compiler** — converts frequently used bytecode into machine code for speed

- **Garbage Collector** — removes unused objects, frees heap memory automatically

4. JNI (Java Native Interface)

Lets Java programs communicate with native code written in C/C++.

5. Native Method Library

Platform-specific libraries (.dll on Windows, .so on Linux) used by native methods.

Memory Trick

Class Loader → loads classes | Runtime Data Area → stores data | Execution Engine → executes bytecode | JNI → connects to native code | Native Libraries → OS-specific functions

11. Garbage Collector (GC)

Automatically removes objects with no active references, freeing heap memory and reducing memory leaks.

- Automatic memory management
- Prevents memory leaks
- Improves application stability

12. Why Do We Need JRE?

- **Provides Runtime Environment** — required to execute Java programs
- **Contains JVM** — converts bytecode into machine code
- **Includes Java Libraries** — built-in classes and APIs needed at runtime
- **Runs Java Applications** — executes .class files without needing javac
- **No Development Tools** — cannot compile source code (no javac)